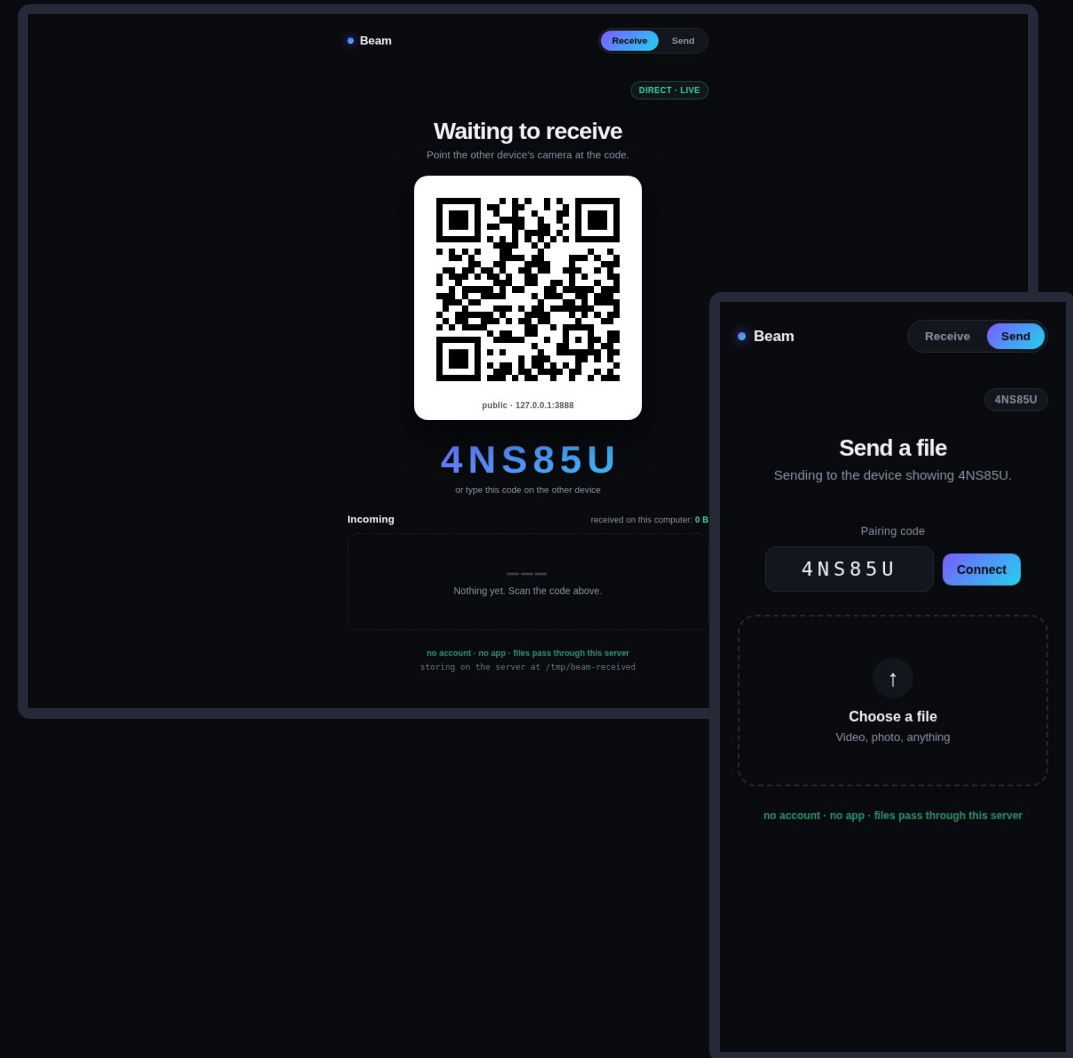


BEAM

Send a file to a computer running Beam.
Scan a code or type one. No sender
account, no sender app.

Sender needs only a browser | Receiver runs the Beam server
(Node.js)



Mixed-device file sharing is still awkward

01

Pairing is the hard part

Sending bytes is easy. Getting two unrelated devices to find each other is where it falls apart.

02

Mixed-device friction

A phone and a laptop from different makers often need a workaround to share one file.

03

Alternatives add steps

Email, chat apps and cloud drives need an account, an app or a round trip through a service.

Beam aims for pairing with almost no thought: scan or type a code, then send.

Pair, send, receive: no sender setup

1 Start the receiver

Run the Beam server (Node.js) on the receiving computer, reachable on the network by the other device. Open it: QR code and 6-character code appear.

2 Pair

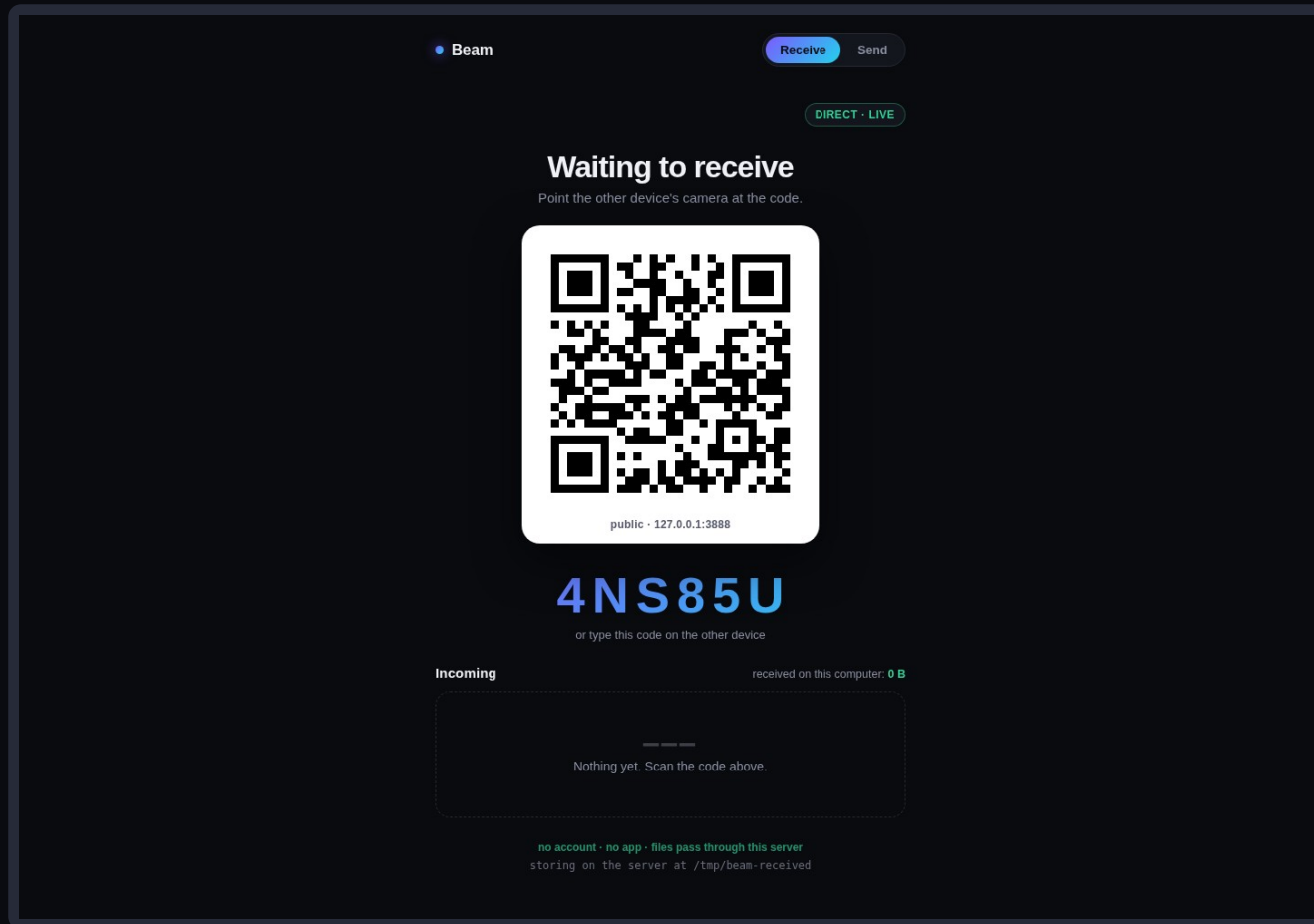
On the sending device, scan the QR with the camera, or type the code on the Send tab. A browser is enough.

3 Send

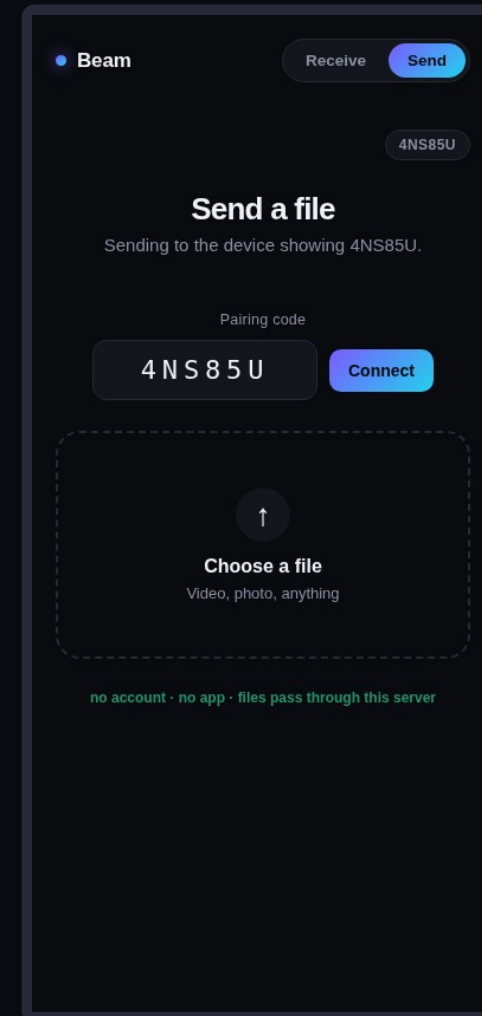
Choose a file. It uploads in chunks and appears on the receiver with live progress.

Two computers? Both computers open the same Beam server. Each screen has Receive and Send tabs, so one can show a code and the other can type it, with no camera. Connect checks the code with the server before you pick a file, so a typo fails immediately.

The receiving screen and the sending screen



Receiver: QR, code, incoming list

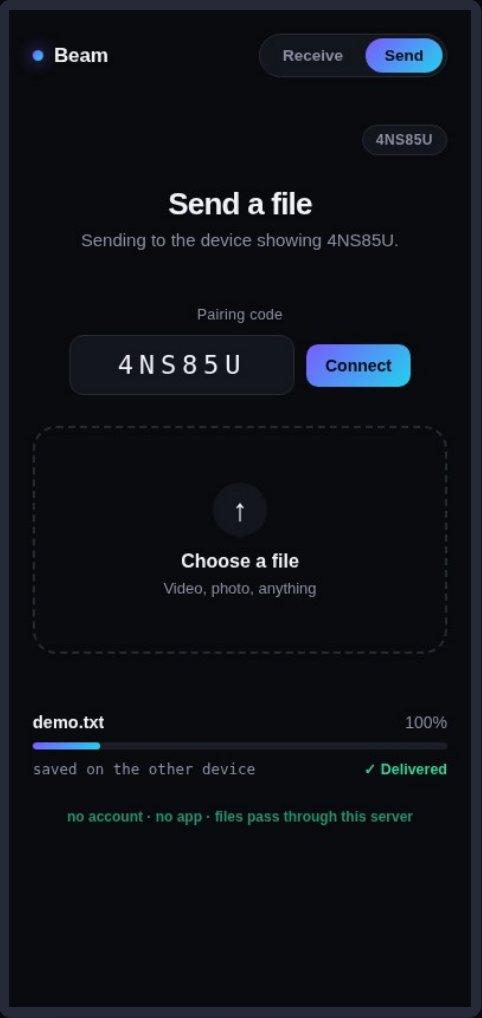


Phone width view,
paired to the code
shown on the left

A file goes in one side and lands on the other



Receiver: demo.txt saved, Download and Show in folder



Sender shows
Delivered

Built to finish the transfer, not just start it



5 MB chunks, pipelined

Uploads go up in 5 MB chunks with several in flight at once, written at fixed offsets.



Live progress

The receiver is updated through server-sent events (SSE). The sender's progress comes from its own browser upload events.



Safe filenames

Directory parts and characters that break paths are stripped; names are capped at 180 characters.



Short-lived sessions

Codes use 6 random characters from an alphabet without look-alikes. Sessions expire after 10 minutes.



Size cap

A configurable limit, 2 GB by default, rejects oversized files before upload.



Image previews

Image files get a thumbnail on the receiver. SVG is excluded from previews.

What we tested, and what we didn't

Verified

- 52 of 54 automated socket tests pass
- Byte-exact chunked upload, including scrambled chunk order
- SSE progress, download, image previews, filename safety
- Browser run: QR and code created, phone-width send page opened, file delivered, bytes equal on disk
- Unknown code rejected; valid code accepted in lower case

Not yet verified

- 2 tests need a LAN network interface, absent in our test environment
- Run was on one machine, not two physical devices
- No real camera scan of the QR yet
- No cross-network or tunnel test
- Files pass through the server; no end-to-end encryption claim

How Beam maps to the judging criteria

Criterion	Pts	Our case
Technical execution	20	Chunked pipelined upload, live progress, path-safe names, size cap, session expiry, 52/54 tests
Innovation & Creativity	15	QR transfer is an existing category; our angle is account-free browser sending with a computer-controlled receiver and code fallback
Impact & Usefulness	20	A familiar problem with plausible users; fewer steps for the sender. Demand is not measured
Completeness & Working Demo	15	Real browser upload, saved file, bytes verified. Browser simulation, not physical phones
Design & UX	10	One screen per role, dark UI, large code, state always visible
Presentation	10	Live demo of the real flow; limits stated plainly
Judge's Preference	10	Personal pick; our part is a direct, easy-to-follow demo

Scan. Send. Done.

Browser-based sending: no sender account, no sender app. Runs from a Node.js server on the receiving computer.

github.com/the-X-alien/beam